DBMS

MODULE V

Single-User versus Multiuser Systems

- **Single-User System**:
- ▫ At most one user at a time can use the system.
- **Multiuser System**:
- ▫ Many users can access the system concurrently.
- **Concurrency**
- ▫ **Interleaved processing**:
  - Concurrent execution of processes is interleaved in a single CPU
- ▫ **Parallel processing**:
  - Processes are concurrently executed in multiple CPUs.

Transaction

- It is a group of database commands that can change/ access the data stored in a database
- Eg: Accounts
    - // Transfer Rs. 100 from Mark's Account to Mary's account

```
BEGIN TRANSACTION
Update Account Set Balance=Balance-100 where Id=1;
Update Account Set Balance=Balance+100 where Id=2
END TRANSACTION
```

Id	Name	Balance
1	Mark	1000
2	Mary	1000

Transactions

- A transaction is treated as a single unit of work
- Transaction maintains integrity of data in a database
- Eg, Transfer Rs. 100 from A to B:
- **Transaction boundaries**:
  - Begin and End transaction.

BEGIN TRANSACTION

R(A); //Read the balance of A

A=A-100;

W(A); //Write the updated value

R(B);
B=B+100;
W(B);

END TRANSACTION

Terminology

- BEGIN TRANSACTION
  - Begining of transaction execution
- END TRANSACTION
  - Transaction execution is completed
- ROLLBACK_TRANSACTION(ABORT)
  - This signals that the transaction has ended unsuccessfully, so that any changes made by that transaction must be undone
- COMMIT_TRANSACTION
  - Signals successful end of the transaction. All changes made by the transaction are recorded permanently in the database and will not be undone

- read_item(X) command
- read_item(X) command includes the following steps:
- ▫ Find the address of the disk block that contains item X.
- ▫ Copy that disk block into a buffer in main memory (if that disk block is not already in some main memory buffer).
- ▫ Copy item X from the buffer to the program variable named X.

- **write_item(X)** command includes the following steps:
  - Find the address of the disk block that contains item X.
  - Copy that disk block into a buffer in main memory (if that disk block is not already in some main memory buffer).
  - Copy item X from the program variable named X into its correct location in the buffer.
  - Store the updated block from the buffer back to disk (either immediately or at some later point in time).
- the decision about when to store a modified disk block whose contents are in a main memory buffer is handled by the recovery manager of the DBMS in cooperation with the underlying operating system.

(a)

T_1

read_item(X);
$X := X - N$;
write_item(X);
read_item(Y);
$Y := Y + N$;
write_item(Y);

(b)

T_2

read_item(X);
$X := X + M$;
write_item(X);

Figure 21.2
Two sample transactions. (a) Transaction T_1. (b) Transaction T_2.

Concurrency problems in DBMS Transactions

- When multiple transactions execute concurrently in an uncontrolled or unrestricted manner, then it might lead to several problems.
- These problems are commonly referred to as concurrency problems in a database environment.

The Lost Update Problem

- This occurs when two transactions that access the same database items have their operations interleaved in a way that makes the value of some database item incorrect.
- an update done to a data item by a transaction is lost as it is overwritten by the update done by another transaction.

Time	T_1	T_2
	read_item(X);	
	$X := X - N$;	
		read_item(X);
		$X := X + M$;
	write_item(X);	
	read_item(Y);	
		write_item(X);
	$Y := Y + N$;	
	write_item(Y);	

Item X has an incorrect value because its update by T_1 is *lost* (overwritten).

- The Temporary Update (or Dirty Read) Problem
  - This occurs when one transaction updates a database item and then the transaction fails for some reason
  - The updated item is accessed by another transaction before it is changed back to its original value.

Time	T_1	T_2
	read_item(X);	
	$X := X - N$;	
	write_item(X);	
		read_item(X);
		$X := X + M$;
		write_item(X);
	read_item(Y);	

Transaction T_1 fails and must change the value of X back to its old value; meanwhile T_2 has read the *temporary* incorrect value of X.

Incorrect Summary Problem

- Consider a situation, where one transaction is applying the aggregate function on some records while another transaction is updating these records.
- The aggregate function may calculate some values before the values have been updated and others after they are updated.

T_1	T_3
	$sum := 0$;
	read_item(A);
	$sum := sum + A$;
	⋮
read_item(X);	
$X := X - N$;	
write_item(X);	
	read_item(X);
	$sum := sum + X$;
	read_item(Y);
	$sum := sum + Y$;
read_item(Y);	
$Y := Y + N$;	
write_item(Y);	

T_3 reads X after N is subtracted and reads Y before N is added; a wrong summary is the result (off by N).

The Unrepeatable Read Problem

- where a transaction T reads the same item twice and the item is changed by another transaction T' between the two reads.
  - Hence, T receives different values for its two reads of the same item.
  - The unrepeatable problem occurs when two or more read operations of the same transaction read
  - different values of the same variable.

T1	T2
Read(X)	
	Read(X)
Write(X)	
	Read(X)

Desirable Properties of a Transaction

- ACID properties:

1. Atomicity
2. Consistency
3. Isolation
4. Durability

Atomicity

- A transaction is said to be atomic if either all of the commands are succeeded or none of them
- Responsibility of Recovery Subsystem
- Eg; Transfer Rs. 100 from A to B:

BEGIN TRANSACTION

R(A);
A=A-100;
W(A);
R(B);
B=B+100;
W(B)

END TRANSACTION

Either execute all command or no command at all

Consistency

- A transaction should be consistency preserving
- It should take the database from one consistent state to another
- Responsibility of Programmers who wrote the database programs
- Eg: Transfer Rs. 100 from A to B

A=1000	→	A=900
B=1000		B=1100
Total=2000		Total=2000

- Isolation
  - Transactions should be isolated to each other during concurrent execution
  - Responsibility: Concurrent control subsystem

- Durability or Permanency
  - The changes applied to the database by committed transaction must persist in the database
  - these changes must not be lost because of any failure
  - Responsibility of Recovery Subsystem

Why **recovery** is needed

- What causes a Transaction to fail
- A computer failure (system crash)
- A transaction or system error
- Local errors or exception conditions detected by the transaction:
- Concurrency control enforcement
- Disk failure
- Physical problems and catastrophes

States Of a Transaction

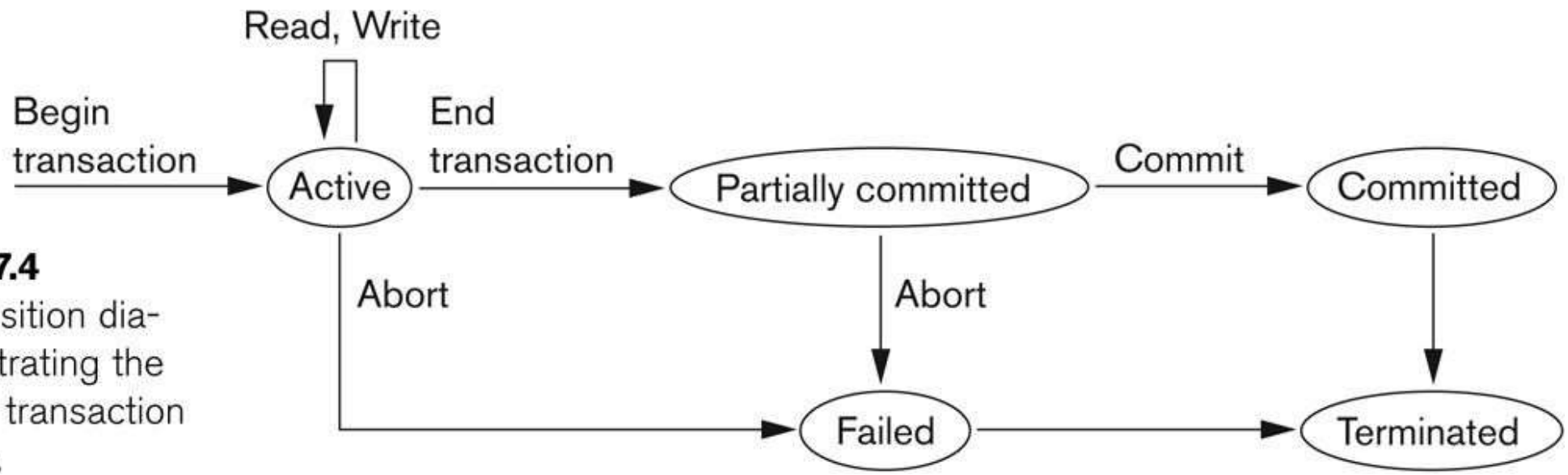


Figure 17.4
State transition diagram illustrating the states for transaction execution.

States of a Transaction

- Active
  - Statements inside the transaction block are executed
- Partially Committed
  - All updates/changes made by the transaction is applied on the data block available in the Main Memory Buffer. It is not yet updated in actual data block(Secondary Storage)
- Committed
  - Whenever the changes made by the transaction are permanently recorded in secondary storage , then we can say that the transaction is committed state. Now the changes cant be undone
- Terminated
  - Corresponds to transaction leaving the system
- Failed
  - Active to Failed: Some Commands are not able to complete
  - Partially Committed to Failed: Failure happens before making permanent changes

- Recovery manager keeps track of the following operations:
  - **begin_transaction**: This marks the beginning of transaction execution.
  - **read or write:** These specify read or write operations on the database items that are executed as part of a transaction.
  - **end_transaction**: This specifies that read and write transaction operations have ended and marks the end limit of transaction execution.
    - At this point it may be necessary to check whether the changes introduced by the transaction can be permanently applied to the database or whether the transaction has to be aborted because it violates concurrency control or for some other reason.
  - **commit_transaction:** This signals a successful end of the transaction so that any changes (updates) executed by the transaction can be safely committed to the database and will not be undone.
  - **rollback (or abort):** This signals that the transaction has ended unsuccessfully, so that any changes or effects that the transaction may have applied to the database must be undone.

The System Log

- ▫ **Log or Journal**:
  - ❖The log keeps track of all transaction operations that affect the values of database items.
    - This information may be needed to permit recovery from transaction failures.
    - The log is kept on disk, so it is not affected by any type of failure except for disk or catastrophic failure.
    - In addition, the log is periodically backed up to archival storage (tape) to guard against such catastrophic failures.

T in the following discussion refers to a unique transaction-id that is generated automatically by the system and is used to identify each transaction:

- Types of log record:

  [start_transaction,T]:

  - Records that transaction T has started execution.

  [write_item,T,X,old_value,new_value]:

  - Records that transaction T has changed the value of database item X from old_value to new_value.

  [read_item,T,X]:

  - Records that transaction T has read the value of database item X.

  [commit,T]:

  - Records that transaction T has completed successfully, and affirms that its effect can be committed (recorded permanently) to the database.

  [abort,T]:

  - Records that transaction T has been aborted.

Recovery using log records

- If the system crashes, we can recover to a consistent database state by examining the log
  - Because the log contains a record of every write operation that changes the value of some database item, it is possible to undo the effect of these write operations of a transaction T by tracing backward through the log and resetting all items changed by a write operation of T to their old_values.
  - We can also redo the effect of the write operations of a transaction T by tracing forward through the log and setting all items changed by a write operation of T (that did not get done permanently) to their new_values.

Log Based Recovery

- The log is a sequence of records.
- Log of each transaction is maintained in some stable storage so that if any failure occurs, then it can be recovered from there.
- If any operation is performed on the database, then it will be recorded in the log.
- But the process of storing the logs should be done before the actual transaction is applied in the database.
- There are two approaches to modify the database:
  1. Deferred database modification
  2. Immediate database modification

Recovery Using Logs

- When the transaction is initiated, then it writes 'start' log.

<Tn, Start>

- When the transaction is finished, then it writes another log to indicate the end of the transaction.

<Tn, Commit>

- When the system is crashed, then the system consults the log to find which transactions need to be undone and which need to be redone.
- If the log contains the record <Ti, Start> and <Ti, Commit> or <Ti, Commit>, then the Transaction Ti needs to be redone.
- If log contains record<T_n, Start> but does not contain the record either <Ti, commit> or <Ti, abort>, then the Transaction Ti needs to be undone.

Deferred database modification

- The deferred modification technique occurs if the transaction does not modify the database until it has committed.
- In this method, all the logs are created and stored in the stable storage, and the database is updated when a transaction commits.
- Also known as No-Undo/Redo
  - In case of a failure,
  - If the log contains the record <Ti, Start> and <Ti, Commit> or <Ti, Commit>, then the Transaction Ti needs to be redone. (Latest Data is saved to the database)
  - If log contains record<T_n, Start> but does not contain the record either <Ti, commit> or <Ti, abort>, then no change is made. Old value is maintained.

example transactions T_0 and T_1 (T_0 executes before T_1):

T_0: **read** (A)	T_1 : **read** (C);
A: = A – 50;	C:-=C- 100;
Write (A);	**write** (C);
read (B);	
B:= B + 50;	
write (B);	

Below we show the log as it appears at three instances of time.

(a)	(b)	(c)
<T_0 start>	<T_0 start>	<T_0 start>
<T_0, A, 950>	<T_0, A, 950>	<T_0, A, 950>
<T_0, B, 2050>	<T_0, B, 2050>	<T_0, B, 2050>
	<T_0 commit>	<T_0 commit>
	<T_1 start>	<T_1 start>
	<T_1, C, 600>	<T_1, C, 600>
		<T_1 commit>

- If log on stable storage at time of crash is as in case:

(a)No redo actions need to be taken

(b) redo(T0) must be performed since <T0 commit> is present

(c)redo(T0) must be performed followed by redo(T1) since

- <T0 commit> and <Ti commit> are present

Immediate database modification:

- The Immediate modification technique occurs if database modification occurs while the transaction is still active.
- In this technique, the database is modified immediately after every operation. It follows an actual database modification.
- Also known as Undo/Redo
  - In case of a failure,
  - If the log contains the record <Ti, Start> and <Ti, Commit> or <Ti, Commit>, then the Transaction Ti needs to be redone. (Latest Data is saved to the database)
  - If log contains record<T_n, Start> but does not contain the record either <Ti, commit> or <Ti, abort>, then the Transaction Ti needs to be undone.(Old value is saved to the database. The new value if already written is over-written with the old value)

Checkpointing

- The checkpoint is a type of mechanism where all the previous logs are removed from the system and permanently stored in the storage disk.
- The checkpoint is like a bookmark. While the execution of the transaction, such checkpoints are marked, and the transaction is executed then using the steps of the transaction, the log files will be created.
- When it reaches to the checkpoint, then the transaction will be updated into the database, and till that point, the entire log file will be removed from the file. Then the log file is updated with the new step of transaction till next checkpoint and so on.
- The checkpoint is used to declare a point before which the DBMS was in the consistent state, and all transactions were committed.

Recovery Using Checkpoint

- The recovery system reads log files from the end to start. It reads log files from T4 to T1.
- Recovery system maintains two lists, a redo-list, and an undo-list.
- The transaction is put into redo state if the recovery system sees a log with <Tn, Start> and <Tn, Commit> or just <Tn, Commit>. In the redo-list and their previous list, all the transactions are removed and then redone before saving their logs.
- The transaction is put into undo state if the recovery system sees a log with <Tn, Start> but no commit or abort log found. In the undo-list, all the transactions are undone, and their logs are removed.

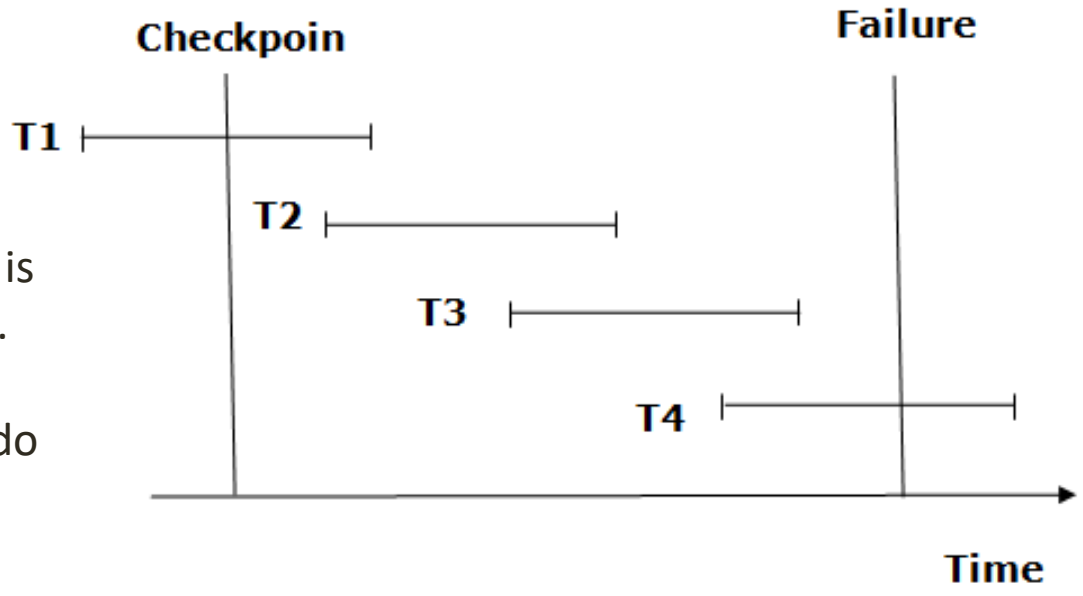
Recovery Using Checkpoint

In the following manner, a recovery system recovers the database from this failure:

For example: In the log file, transaction T2 and T3 will have <Tn, Start> and <Tn, Commit>.

The T1 transaction will have only <Tn, commit> in the log file. That's why the transaction is committed after the checkpoint is crossed. Hence it puts T1, T2 and T3 transaction into redo list.

Transaction T4 will have <Tn, Start>. So T4 will be put into undo list since this transaction is not yet complete and failed amid.



Schedule

- A schedule S of n transactions T1, T2, T3,…Tn is an ordering of operation of the transactions.
- When transactions are executing in an inter-leaved fashion, the order of execution of operations from the various transactions forms what is known as a transaction schedule.

1. Serial schedule
2. Non serial schedule
3. Serializable schedule

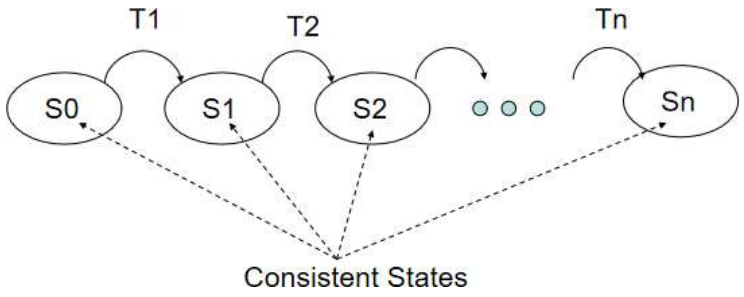
Serial Schedule

- Entire transactions are performed in serial order
- Only One transaction is active at a time
- Serial schedule : T1 T2

T1	T2
R(A)	
A=A+50	
R(B)	
B=B-30	
W(B)	
	R(A) A=A+40 W(A)
	R(B)
	B=B-60 W(B)

Initially	Final Value
A=100	A=190
B=200	B=110

- Every serial schedule is considered as correct if the transactions are independent
- it does not matter which transaction is executed first
- T1 T2 T3…Tn =T2T3T4T1… TN=T3Tn T1 T4 T6…T10
- All these serial schedules are correct schedules



Problems with serial schedules

1. Poor resource utilization
2. More waiting time
3. Less throughput
4. Late response

Serial schedules are unacceptable in practice even though they are correct

- Serial schedules are always
  - Consistent
  - Recoverable
  - Cascadeless
  - Strict

Non Serial Schedules

- Executing the transaction in an interleaved or instructions of transactions are interleaved. Schedule S1

T1	T2
R(A) A=A+50	
W(A)	
	R(A)
	A=A+40
	W(A)
R(B)	
B=B-30	
W(B)	
	R(B) B=B-60 W(B)

Initially	Final Value
A=100	A=190
B=200	B=110

Schedule S1 gives the correct result same as serial schedule

Schedule S2

T1	T2
R(A) A=A+50	
	R(A)
	A=A+40
	W(A)
W(A)	
R(B)	
B=B-30	
	R(B)
	B=B-60
	W(B)
W(B)	

Initially	Final Value	
A=100	A=150	B=170
B=200		

Schedule S2 gives the erroneous result

not same as serial schedule

- Serial Schedules always gives correct result
- But there are some drawbacks
- In order to avoid that we introduced non serial schedules
- But not all non serial schedules are correct
- So we need a non serial schedule which always give a correct result
- or Equivalent to a Serial Schedule = Serializable Schedule

- Non-serial schedules are **NOT** always
  - Consistent
  - Recoverable
  - Cascadeless
  - Strict
- It can be of two types namely, Serializable and Non-Serializable Schedule.

Serializable Schedule

- A non serial schedule of n transactions is said to be serializable if it is equivalent to some serial schedule of same n transactions

- Every serializable schedule is considered as correct

- This is used to maintain the consistency of the database.
- It is mainly used in the Non-Serial scheduling to verify whether the scheduling will lead to any inconsistency or not.

- Some non-serial schedules may lead to inconsistency of the database.
- Serializability is a concept that helps to identify which non-serial schedules are correct and will maintain the consistency of the database.
- If a given non-serial schedule of ‘n’ transactions is equivalent to some serial schedule of ‘n’ transactions, then it is called as a serializable schedule.

Serial Schedules	Serializable Schedules
No concurrency is allowed.Thus, all the transactions necessarily execute serially one after the other.	Concurrency is allowed. Thus, multiple transactions can execute concurrently.
Serial schedules lead to less resource utilization and CPU throughput.	Serializable schedules improve both resource utilization and CPU throughput.
Serial Schedules are less efficient as compared to serializable schedules.	Serializable Schedules are always better than serial schedules.

Serializability

- Serializability is mainly of two types
  - Conflict Serializability
  - View Serializability

Conflict Serializability

- If a given non-serial schedule can be converted into a serial schedule by swapping its non-conflicting operations, then it is called as a **conflict serializable schedule**.

- Two operations are called as **conflicting operations** if all the following conditions hold true for them-
  - Both the operations belong to different transactions
  - Both the operations are on the same data item
  - At least one of the two operations is a write operation

- Conflicting Operations

T1	T2
Read(A)	
	Read(A)

Swapped ⇨

T1	T2
	Read(A)
Read(A)	

Schedule S1 **Schedule S2**

Here, S1 = S2. That means it is non-conflict.

T1	T2
Read(A)	
	Write(A)

Swapped ⇨

T1	T2
	Write(A)
Read(A)	

Schedule S1 **Schedule S2**

Here, S1 ≠ S2. That means it is conflict

Conflict Equivalent

- In the conflict equivalent, one can be transformed to another by swapping non-conflicting operations.
- Two schedules are said to be conflict equivalent if and only if:
  - They contain the same set of the transaction.
  - If each pair of conflict operations are ordered in the same way.

- Conflict Equivalent

T1	T2
Read(A) Write(A)	
	Read(A) Write(A)
Read(B) Write(B)	
	Read(B) Write(B)

Schedule S1

T1	T2
Read(A) Write(A) Read(B) Write(B)	
	Read(A) Write(A) Read(B) Write(B)

Schedule S1 can be transformed into a serial schedule by swapping non-conflicting operations of S1.

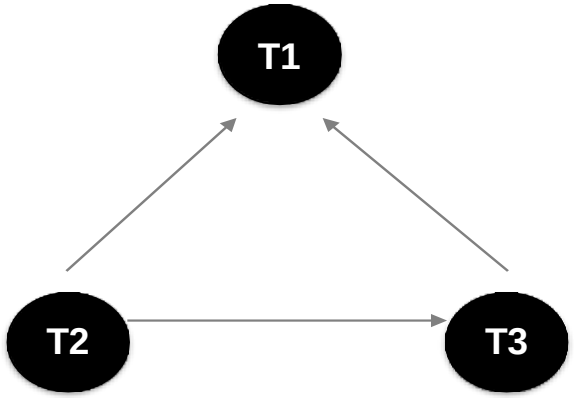
Testing for Conflict Serializability of a Schedule

- There is a simple algorithm for determining whether a particular schedule is conflict serializable or not.
- The Algorithm can be written as:
  1. Create a node T in the graph for each participating transaction in the schedule.
  2. For the conflicting operation read_item(X) and write_item(X) - If a Transaction Tj executes a read_item (X) after Ti executes a write_item (X), draw an edge from Ti to Tj in the graph.
  3. For the conflicting operation write_item(X) and read_item(X) - If a Transaction Tj executes a write_item (X) after Ti executes a read_item (X), draw an edge from Ti to Tj in the graph.
  4. For the conflicting operation write_item(X) and write_item(X) - If a Transaction Tj executes a write_item (X) after Ti executes a write_item (X), draw an edge from Ti to Tj in the graph.
  5. The Schedule S is serializable if there is no cycle in the precedence graph.

Testing for Conflict Serializability of a Schedule – Example

T1	T2	T3
R(x)		
		R(y)
		R(x)
	R(y)	
	R(z)	
		W(y)
	W(z)	
R(z)		
W(x)		
W(z)		

- Draw Edge when we have
- read_item(X) and write_item(X)
- write_item(X) and read_item(X)
- write_item(X) and write_item(X)



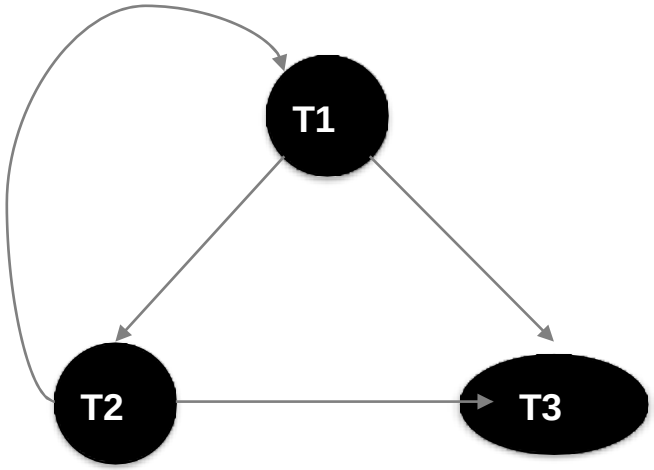
- T1-R(x): no W(x) in T2, T3
- T3-R(y): no W(y) in T2, T1
- T3-R(x): W(x) in T1 draw the edge T3->T1
- T2-R(y): W(y) in T3 draw the edge T2->T3
- T2-R(z): W(z) in T1 draw the edge T2->T1
- T3-W(y): no R(y) or W(y) in T2, T1
- T2-W(z): R(z) and W(z) in T1 draw edge T2->T1(already there)

- As we have no Cycle/Loop in the Precedence Graph these schedule is conflict serializable

Example 2

T1	T2	T3
R(A)		
	W(A)	
W(A)		
		W(A)

- Draw Edge when we have
- read_item(X) and write_item(X)
- write_item(X) and read_item(X)
- write_item(X) and write_item(X)

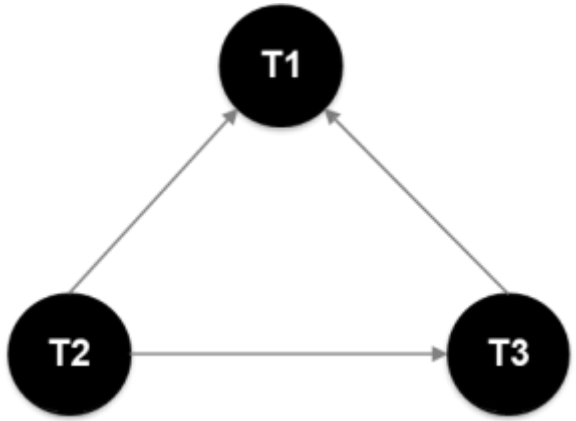


- As we have Cycle/Loop in the Precedence Graph this schedule is Non conflict serializable . Is this serializable?

Previous university question

Check if the following schedules are conflict-serializable using precedence graph. If so, give the equivalent serial schedule(s). r3(X), r2(X), w3(X), r1(X), w1(X). (Note: ri(X)/wi(X) means transaction Ti issues read/write on item X.)

T1	T2	T3
		R(x)
	R(x)	
		W(x)
R(x) W(x)		



- Draw Edge when we have
- read_item(X) and write_item(X)
- write_item(X) and read_item(X)
- write_item(X) and write_item(X)

- As we have no Cycle/Loop in the Precedence Graph this schedule is conflict serializable .

Conflict Serializability Contd…

- By performing the **Topological Sort** of the **Directed Acyclic Graph** so obtained, the corresponding serial schedule(s) can be found.
- Such schedules can be more than 1.

View Serializability

- View Serializability is a process to find out that a given schedule is view serializable or not.
- To check whether a given schedule is view serializable, we need to check whether the given schedule is View Equivalent to its serial schedule.
- If a given schedule is found to be view equivalent to some serial schedule, then it is called as a view serializable schedule.

View Equivalent- Conditions

- Two schedules S1 and S2 are said to be view equivalent if they satisfy the following conditions:
- Condition - 01
  - For each data item X, if transaction T_i reads X from the database initially in schedule S1, then in schedule S2 also, T_i must perform the initial read of X from the database.
  - "**Initial readers must be same for all the data items**"

- Suppose two schedule S1 and S2. In schedule S1, if a transaction T1 is reading the data item A, then in S2, transaction T1 should also read A.

T1	T2
Read(A)	
	Write(A)

Schedule S1

T1	T2
	Write(A)
Read(A)	

Schedule S2

Initial read operation in S1 is done by T1 and in S2 it is also done by T1.

View Equivalent- Conditions

- Condition - 02
  - If transaction T_i reads a data item that has been updated by the transaction T_j in schedule S1, then in schedule S2 also, transaction T_i must read the same data item that has been updated by the transaction T_j.
  - **"Write-read sequence must be same."**

In schedule S1, if Ti is reading A which is updated by Tj then in S2 also, Ti should read A which is updated by Tj.

T1	T2	T3
Write(A)		
	Write(A)	
		Read(A)

Schedule S1

T1	T2	T3
Write(A)		
	Write(A)	
		Read(A)

Schedule S2

These two schedules are not view equal because, in S1, T3 is reading A updated by T2 and in S2, T3 is reading A updated by T1.

View Equivalent- Conditions

- Condition - 03
  - For each data item X, if X has been updated at last by transaction T_i in schedule S1, then in schedule S2 also, X must be updated at last by transaction T_i.
  - "**Final writers must be same for all the data items**".

- A final write must be the same between both the schedules. In schedule S1, if a transaction T1 updates A at last then in S2, final writes operations should also be done by T1.

T1	T2	T3
Write(A)		
	Read(A)	
		Write(A)

Schedule S1

T1	T2	T3
Write(A)	Read(A)	
		Write(A)

Schedule S2

Final write operation in S1 is done by T3 and in S2, the final write operation is also done by T3.

View Serializability Contd…

- All conflict serializable schedules are view serializable.
- All view serializable schedules may or may not be conflict serializable.
- For those schedules that are not conflict serializable, if there are No blind writes (Writing without reading is called as a blind write) means not a view serializable schedule.

- Draw dependency graph, check for cycle.

Example

Non-Serial

S1

T1	T2
R(X)	
W(X)	
	R(X)
	W(X)
R(Y)	
W(Y)	
	R(Y)
	W(Y)

Serial

S2

T1	T2
R(X)	
W(X)	
R(Y)	
W(Y)	
	R(X)
	W(X)
	R(Y)
	W(Y)

Beginnerbook.com

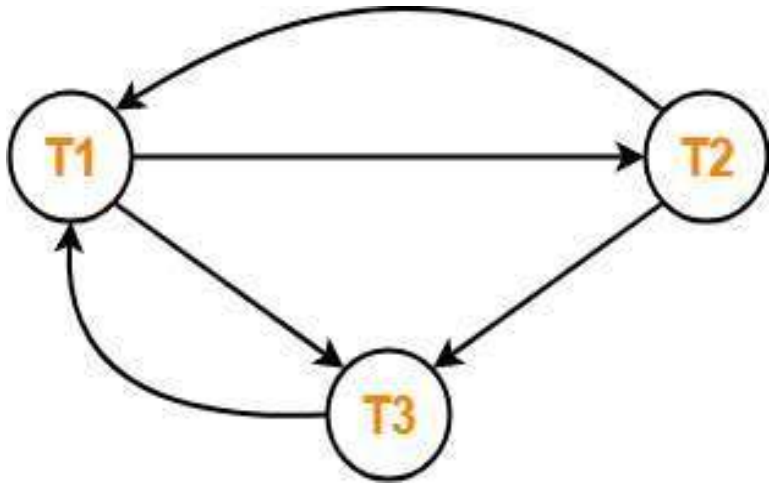
S2 is the serial schedule of S1. If we can prove that they are view equivalent then we we can say that given schedule S1 is view Serializable

Another Practice Question

- Check whether the given schedule S is view serializable or not. If yes, then give the serial schedule. **S : $R_1(A)$, $W_2(A)$, $R_3(A)$, $W_1(A)$, $W_3(A)$**
  - $R_1(A), W_2(A) (T_1 \rightarrow T_2)$
  - $R_1(A), W_3(A) (T_1 \rightarrow T_3)$
  - $W_2(A), R_3(A) (T_2 \rightarrow T_3)$
  - $W_2(A), W_1(A) (T_2 \rightarrow T_1)$
  - $W_2(A), W_3(A) (T_2 \rightarrow T_3)$
  - $R_3(A), W_1(A) (T_3 \rightarrow T_1)$
  - $W_1(A), W_3(A) (T_1 \rightarrow T_3)$

T1	T2	T3
R (A)		
	W (A)	
		R (A)
W (A)		
		W (A)

- $R_1(A)$, $W_2(A)$ $(T_1 \rightarrow T_2)$
- $R_1(A)$, $W_3(A)$ $(T_1 \rightarrow T_3)$
- $W_2(A)$, $R_3(A)$ $(T_2 \rightarrow T_3)$
- $W_2(A)$, $W_1(A)$ $(T_2 \rightarrow T_1)$
- $W_2(A)$, $W_3(A)$ $(T_2 \rightarrow T_3)$
- $R_3(A)$, $W_1(A)$ $(T_3 \rightarrow T_1)$
- $W_1(A)$, $W_3(A)$ $(T_1 \rightarrow T_3)$
- The given schedule S is not conflict serializable, so, it may or may not be view serializable.



- There exists a blind write W_2 (A) in the given schedule S.
- Therefore, the given schedule S may or may not be view serializable.

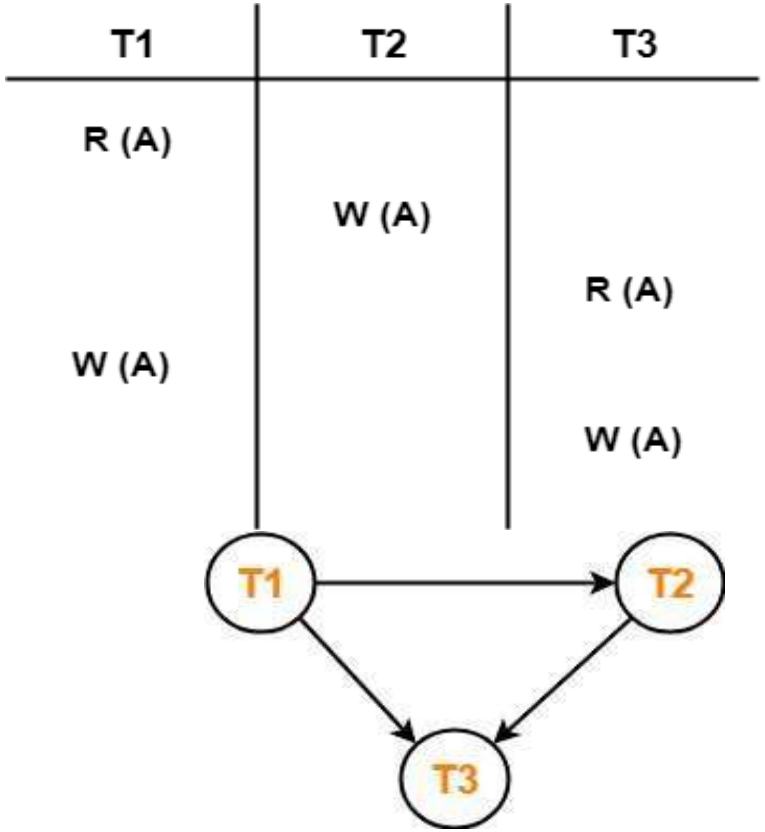
T1	T2	T3
R (A)		
	W (A)	
		R (A)
W (A)		
		W (A)

- T1 firstly reads A and T2 firstly updates A.
- So, T1 must execute before T2.
- Thus, we get the dependency **T1→ T2**.
- Final updation on A is made by the transaction T3.

T1	T2	T3
R (A)		
	W (A)	
		R (A)
W (A)		
		W (A)

- So, T3 must execute after all other transactions.
- Thus, we get the dependency **(T1, T2)→ T3**.
- From write-read sequence, we get the dependency **T2 → T3**

- Clearly, there exists no cycle in the dependency graph.
- Therefore, the given schedule S is view serializable.
- The serialization order T1 → T2 → T3.



Recoverable Schedules

- Schedules in which transactions commit only after all transactions whose changes they read commit are called recoverable schedules.
  - If some transaction T_j is reading value updated or written by some other transaction T_i,
    - then the commit of T_j must occur after the commit of T_i.

T_1	T_2
R(A)	
W(A)	
	W(A)
	R(A)
commit	
	commit

This is a recoverable schedule since T_1 commits before T_2, that makes the value read by T_2 correct.

Irrecoverable Schedule

T1	T1's buffer space	T2	T2's Buffer Space	Database
				A=5000
R(A);	A=5000			A=5000
A=A-100;	A=4000			A=5000
W(A);	A=4000			A=4000
		R(A);	A=4000	A=4000
		A=A+500;	A=4500	A=4000
		W(A);	A=4500	A=4500
		Commit;		
Failure Point				
Commit;				

- The table shows a schedule with two transactions, T_1 reads and writes A and that value is read and written by T_2.
- T_2 commits. But later on, T_1 fails.
- So we have to rollback T_1. Since T_2 has read the value written by T_1, it should also be rollbacked.
- But we have already committed that.
- So this schedule is irrecoverable schedule.
- When T_j is reading the value updated by T_i and T_j is committed before committing of T_i, the schedule will be irrecoverable.

Note:

- If the given schedule is conflict serializable, then it is surely recoverable. Stop and report your answer.
- If there does not exist any dirty read operation, then the schedule is surely recoverable. Stop and report your answer.
- If the commit operation of the transaction performing the dirty read is delayed till the commit or abort operation of the transaction which updated the value, then the schedule is recoverable.

Recoverable with Cascading Rollback

- Cascading Rollback (or cascading abort) occur in some recoverable schedules, where an *uncommitted transaction has to be rolled back because it read an item from a transaction that failed*.

T1	T1's buffer space	T2	T2's Buffer Space	Database
				A=5000
R(A);	A=5000			A=5000
A=A-100;	A=4000			A=5000
W(A);	A=4000			A=4000
		R(A);	A=4000	A=4000
		A=A+500;	A=4500	A=4000
		W(A);	A=4500	A=4500
Failure Point				
Commit;				
		Commit;		

- The table shows a schedule with two transactions, T_1 reads and writes A and that value is read and written by T_2.
- But later on, T_1 fails. So we have to rollback T_1.
- Since T_2 has read the value written by T_1, it should also be rollbacked.
- As it has not committed, we can rollback T_2 as well. So it is recoverable with cascading rollback.
- *If T_j is reading value updated by T_i and commit of T_j is delayed till commit of T_i, the schedule is called recoverable with cascading rollback.*

T1	T2	T3	T4
R (A)			
W (A)			
	R (A)		
	W (A)		
		R (A)	
		W (A)	
			R (A)
			W (A)
Failure			

Cascading Recoverable Schedule

Cascadeless Recoverable Rollback

- A schedule is said to be cascadeless, or to avoid cascading rollback, *if every transaction in the schedule reads only items that were written by committed transactions.*

T1	T1's buffer space	T2	T2's Buffer Space	Database
				A=5000
R(A);	A=5000			A=5000
A=A-100;	A=4000			A=5000
W(A);	A=4000			A=4000
Commit;				
		R(A);	A=4000	A=4000
		A=A+500;	A=4500	A=4000
		W(A);	A=4500	A=4500
		Commit;		

- The table shows a schedule with two transactions, T1 reads and writes A and commits and that value is read by T2.
- But if T1 fails before commit, no other transaction has read its value, so there is no need to rollback other transaction.
- So this is a Cascadeless recoverable schedule.
- *If Tj reads value updated by Ti only after Ti is committed, the schedule will be cascadeless recoverable.*

T1	T2	T3
R (A)		
W (A)		
Commit		
	R (A)	
	W (A)	
	Commit	
		R (A)
		W (A)
		Commit

Cascadeless Schedule

- It avoids cascading roll back and thus saves CPU time.

Strict Schedule

- If in a schedule, a transaction is neither allowed to read nor write a data item until the last transaction that has written it is committed or aborted, then such a schedule is called as a **Strict Schedule**.
- In other words,
  - Strict schedule allows only committed read and write operations.
  - Clearly, strict schedule implementsmore restrictions than cascadeless schedule.

Concurrency Control Protocols

- In order to acheive serializability and to prevent the occurence of concurrency problems in non serial schedules we are using the concurrency control protocols

- Three type of Concurrency Control Protocols

1. **Lock based Protocols**
2. Time stamp based Protocol
3. Graph based protocols

Lock Based Protocols

- Locking protocol means, whenever a transaction wants to perform any operation(Read/Write) on any data item, first it should request for lock on that data item.
- Once it acquire the lock then only it can perform the operation
- Locking mechanism is managed by Concurrency Control Subsystem

- **Transaction T1 wants to perform an operation on data item A**
  - Apply lock on that data item(A)
  - **Perform the operation on A**

Types of locks

- Two type of Locks are

1. Shared Lock(S(A))
2. Exclusive Lock(X(A))

Shared Lock

- If a transaction wants to perform read operation on a data item, it should apply shared lock on that item

T1

- S(A)
- Read(A)

Types of locks

- Two type of Locks are

1. Shared Lock(S(A))
2. Exclusive Lock(X(A))

Exclusive Lock

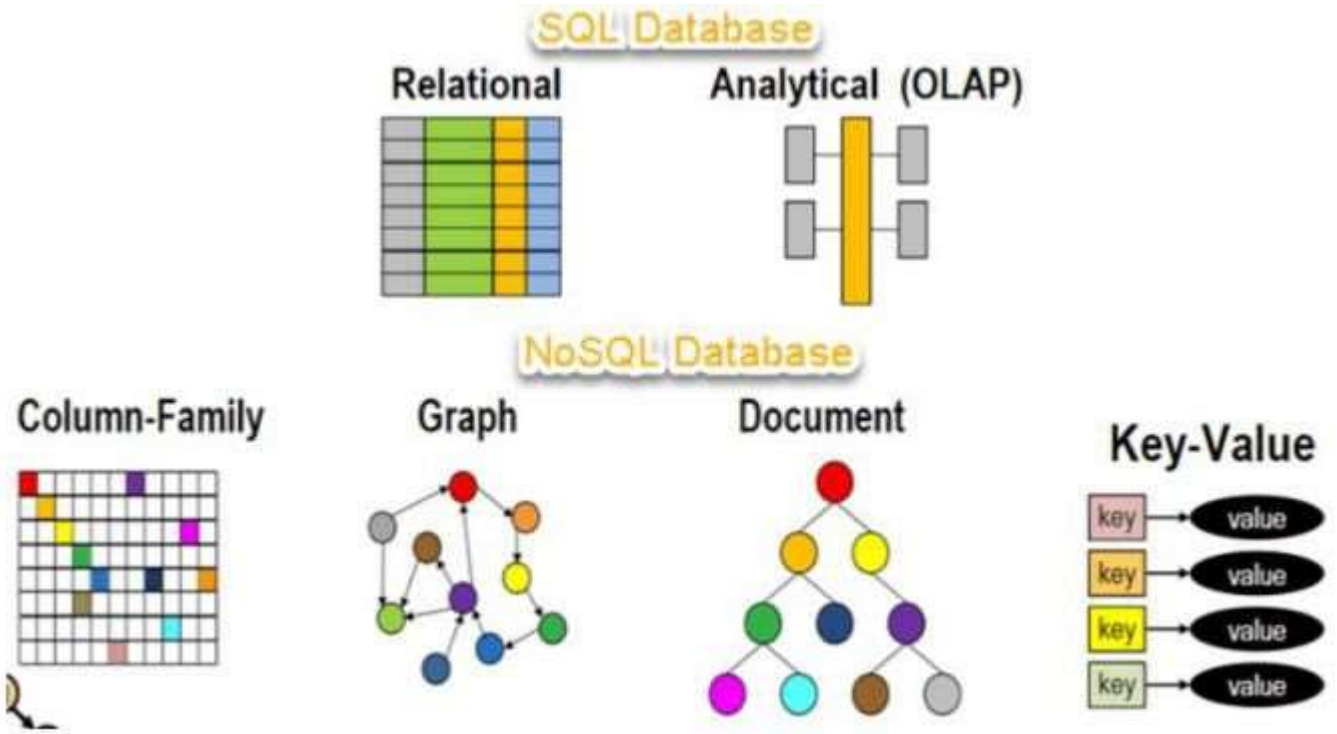
- If a transaction acquire this lock, it can perform read/write/both operation on that data item

T1

- X(A)
- Read(A)
- Write(A)

NoSQL

- NoSQL database stands for “Not Only SQL” or “Not SQL.”
- It is a non-relational Data Management System, that does not require a fixed schema.
- It avoids joins, and is easy to scale. The major purpose of using a NoSQL database is for distributed data stores with humongous data storage needs.
- NoSQL is used for Big data and real-time web apps.
- For example, companies like Twitter, Facebook and Google collect terabytes of user data every single day


SQL Database
Relational
Analytical (OLAP)
NoSQL Database
Column-Family
Graph
Document
Key-Value
key
value
key
value
key
value
key
value

- The concept of NoSQL databases became popular with Internet giants like Google, Facebook, Amazon, etc. who deal with huge volumes of data.
- The system response time becomes slow when you use RDBMS for massive volumes of data.
- To resolve this problem, we could “scale up” our systems by
- upgrading our existing hardware.
- This process is expensive.
- The alternative for this issue is to distribute database load on multiple hosts whenever the load increases. This method is known as “scaling out.”
- NoSQL database is non-relational, so it scales out better than relational databases as they are designed with web applications in mind.

Features of NoSQL

- Non-relational
- ▫ NoSQL databases never follow the relational model
- ▫ Never provide tables with flat fixed-column records
- ▫ Work with self-contained aggregates or BLOBs
- ▫ Doesn’t require object-relational mapping and data normalization
- ▫ No complex features like query languages, query planners,referential
- integrity joins,
- ACID
- ▫ Schema-free
- ▫ NoSQL databases are either schema-free or have relaxed schemas
- ▫ Do not require any sort of definition of the schema of the data
- ▫ Offers heterogeneous structures of data in the same domain

Types of NoSQL

- NoSQL Databases are mainly categorized into four types:
- ▫ Key-value Pair Based
- ▫ Column-oriented
- ▫ Graphs based
- ▫ Document-oriented

Key Value Database

- A key-value database (sometimes called a key-value store) uses a simple key-value method to store data.
- These databases contain a simple string (the key) that is always unique and an arbitrary large data field (the value).
- They are easy to design and implement.

Phone directory

Key	Value
Paul	(091) 9786453778
Greg	(091) 9686154559
Marco	(091) 9868564334

MAC table

Key	Value
10.94.214.172	3c:22:fb:86:c1:b1
10.94.214.173	00:0a:95:9d:68:16
10.94.214.174	3c:1b:fb:45:c4:b1

Column Oriented

- While a relational database stores data in rows and reads data row by row, a column store is organized as a set of columns.
- When you want to run analytics on a small number of columns, you can read those columns directly without consuming memory with the unwanted data.
- Columns are often of the same type and benefit from more efficient compression, making reads even faster.
- Columnar databases can quickly aggregate the value of a given column (adding up the total sales for the year, for example). Use cases include analytics.

Document Oriented Database

- Is a modernized way of storing data as JSON rather than basic columns/rows — i.e. storing data in its native form.
- This storage system lets you retrieve, store, and manage document oriented information
- It's a very popular category of modern NoSQL databases, used by the likes of MongoDB, Cosmos DB, DocumentDB, SimpleDB, PostgreSQL, OrientDB, Elasticsearch and RavenDB.

Graph Based NoSQL

- Graph databases are generally straightforward in how they're structured though. They primarily are composed of two components:
- The Node

▫ This is the actual piece of data itself.

▫ It can be the number of viewers of a youtube video, the number of people who have read a tweet, or it could even be basic information such as people's names, addresses, and so forth.

- The Edge

▫ This explains the actual relationship between two nodes.

▫ Interestingly enough, edges can also have their own pieces of information, such as the nature of the relation between two nodes. Similarly, edges might also have directions describing the flow of said data.